

SLR(1) Parsing Table

①
 $E \rightarrow T + E / T$
 ②
 $T \rightarrow id$
 ③

$E \rightarrow E$
 $E \rightarrow T + E$
 $E \rightarrow T$
 $T \rightarrow id$

I_0

$E \rightarrow \cdot E$
 $E \rightarrow \cdot T + E$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot id$

$E \xrightarrow{+} E \rightarrow E \cdot$
 $T \xrightarrow{+} E \rightarrow T \cdot + E$
 $E \rightarrow T \cdot$
 $T \xrightarrow{id} T \rightarrow id \cdot$

I_1 (accepted)

I_4
 $E \rightarrow T + \cdot E$
 $E \rightarrow \cdot T + E$
 $E \rightarrow \cdot T$
 $T \rightarrow \cdot id$

$E \rightarrow T + E \cdot$

Reduced Term₁

I_5

Reduced Term₃

Step 1 : Follow $T \rightarrow id$

	Follow
$E \rightarrow T + E / T$	$\{ \$ \}$
$T \rightarrow id$	$\{ +, \$ \}$

	Action (terminal)			Go-to (variable)	
	id	+	\$	E	T
I_0	S_3			1	2
I_1			Accept		
I_2		S_4	R_2		
I_3		R_3	R_3		
I_4	S_3			5	2
I_5			R_1		

Reduced નિયમોના ઉપર, follow અથવા shift નામના terminal/ action ટાંચાશે.

$T \rightarrow id$. (Reduced Term₃)

$f(T) \rightarrow \{+, \$\}$

$E \rightarrow T + E$. (Reduced Term₁)

$f(E) \rightarrow \{\$ \}$

CLR(1) Parsing Table

8. ଯଦ୍ୟଦି ତଳି canonical Item ଲାଗି CLR

Q. ଗଢ଼ା ?

$S \rightarrow AA$

$A \rightarrow aA \mid b$

$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA \mid b$

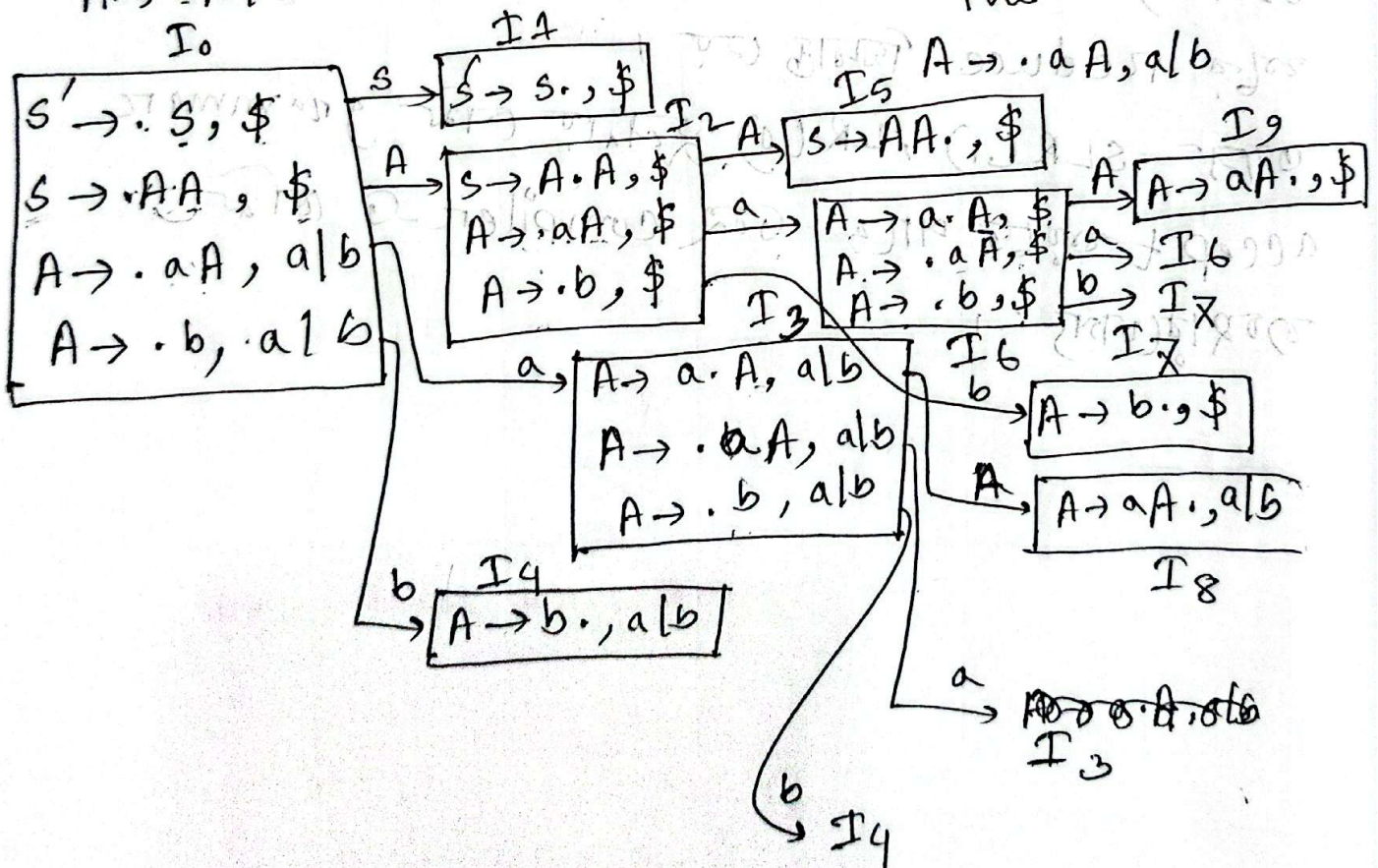
ଉପରୋକ୍ତ ଗ୍ରାମର ଗଢ଼ା
ତା ଗଢ଼ା ଦିଅନ୍ତୁ ଯାହା first
of ଗ୍ରାମ ଗଢ଼ା ଦିଅନ୍ତୁ

$S \rightarrow \cdot AA, \$$

$A, \$$

$A \rightarrow a \mid b$

the



Parsing table

(Item)	Action (Terminal)			Goto (Variable)	
	a	b	\$	S	A
I_0	S_3	S_4		1	2
I_1			Accept		5
I_2	S_6	S_7			5
I_3	S_3	S_4			8
I_4	R_3	R_3			
I_5			R_1		
I_6	S_6	S_7			9
I_7			R_3		
I_8	R_2	R_2			
I_9			R_2		

Reduced G Lookahead G par nro of Terminal.

G cell G nro 1

CLR table merge nro LALR table create nro 1

input same rak G nro merging nro (nro I_3 & I_6)
G input same rak merge nro 1

LR(0) items

I	Action			Goto	
	a	b	\$	S	A
I_0, I_6	S ₃	S ₄			8, 9
I_4, I_7	R ₃	R ₃	R ₃		
I_8, I_9	R ₂	R ₂	R ₂		

Now convert LALR

I	Action			Goto	
	a	b	\$	S	A
I_0	S ₃	S ₄		1	
I_4			AC		
I_2	S ₆	S ₈		5	
I_3, I_6	S ₃	S ₄			8, 9
I_4, I_7	R ₃	R ₃	R ₃		
I_5			R ₁		
I_8, I_9	R ₂	R ₂	R ₂		

5.

No	Basis	LL(1)	LR(0)	SLR	LALR	CLR/LR(1)
1.	Parsing Approach	Top-down	Bottom-up	Bottom-up	Bottom-up	Bottom-up
2.	Derivation used	Leftmost	Rightmost (reverse)	Rightmost (reverse)	Rightmost (reverse)	Rightmost (reverse)
3.	Lookahead	1 symbol	No Lookahead	Uses follow set	Uses Lookahead	Uses full lookahead
4.	Grammar Power	Weak	Weak	Medium	High	Highest
5.	Grammar Accepted	Very few grammars	Few grammars	More than LR(0)	Almost all practical grammars	Almost all grammars
6.	Parsing Table size	Small	Small	Medium	Medium	Very large
7.	Conflicts	Many conflicts	Many conflicts	few conflicts	Very few conflicts	Almost no conflicts
8.	Practical use	Simple languages	Rarely used	Limited use	Widely used (VPL)	Rarely used (large table)

6. SDT $\rightarrow$ Application of SDT
infix to post, post to prefix
parse tree solve (Bottom up approach, top down app.)

Syntax-Directed Translation (SDT)

Syntax-Directed Translation involves associating semantic actions with the production rules of grammar.

Each rule in a grammar is linked to specific actions that are executed when the rule is applied during parsing.

Grammar + Semantic Rules = SDT

Applications of SDT

1. Expression Translation

- converts infix $\rightarrow$ postfix
- converts postfix $\rightarrow$ prefix
- used in expression evaluation

2. Syntax Tree / Parse Tree Construction

- $\rightarrow$ Builds parse tree using grammar rules
- Helps in understanding program structure

3. Intermediate Code Generation

- Generates three-address code
- used between parsing and code optimization

4. Type checking

- Checks type compatibility (ints, float, etc)
- Detects type mismatch error at compile time

5. Declaration Processing

- Processes variable and function declarations
- Inserts information into symbol table

6. Semantic Analysis

- Enforces semantic rules of the language
- Checks scope, usage, and meaning of statements

7. Control statement Translation

- Translates if, if-else, while, for statements
- Generates labels and jump instructions

8. Code optimization Support

- Helps detect redundant operations
- Supports constant folding and simplification

SDT is used to attach semantic meaning to syntax rules for expression translation, code generation, type checking and semantic analysis.

Grammar
 $E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

Rules

$\{ E.val = E.val + T.val \}$

$\{ E.val = T.val \}$

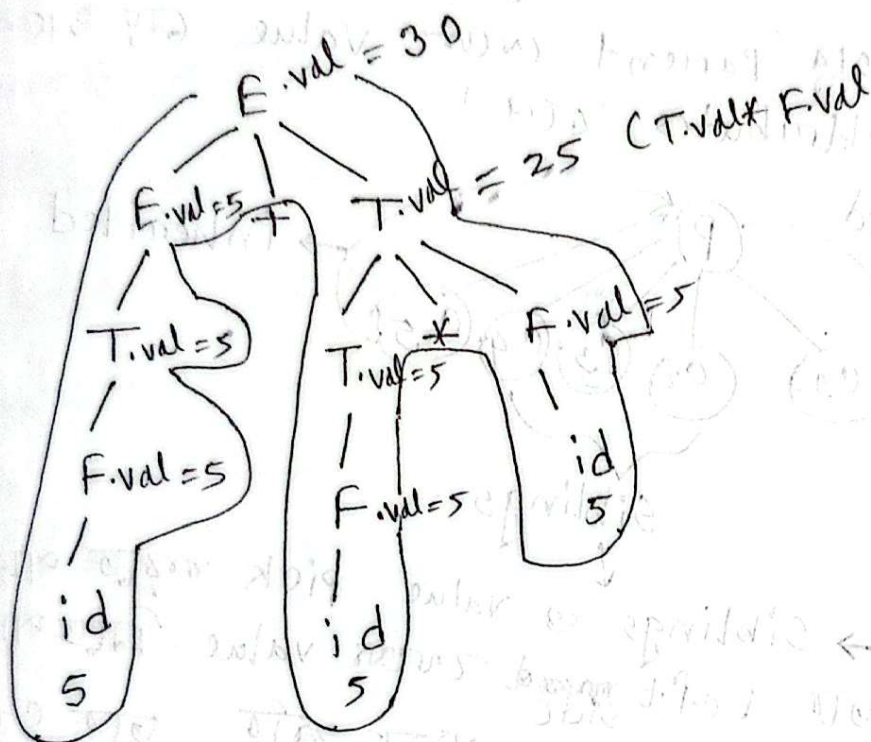
$\{ T.val = T.val * F.val \}$

$\{ T.val = F.val \}$

$\{ F.val = id \}$

Given String:

$5 + 5 * 5 = 30$

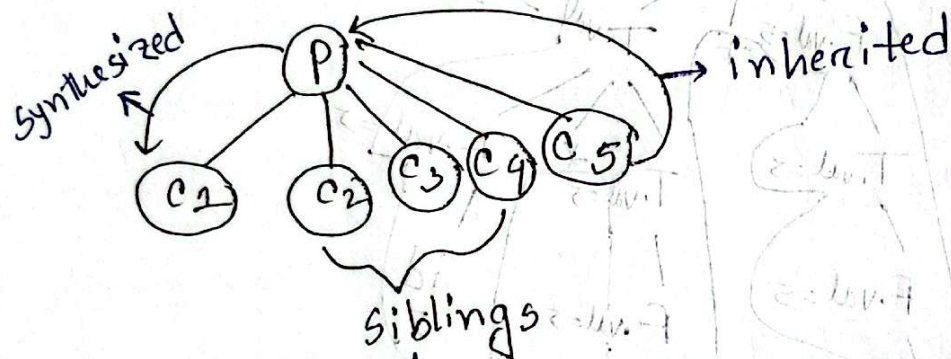


Types of SDT

S-Attributed

L - Attributed

- Parent যদি তার child এর value pick করে তার synthesized attributes বলে।
- child যদি তার parent এর value তার inherited attributes বলে।



- Siblings value pick ନିଅନ୍ତୁ
ତା Left side value ନିଅନ୍ତୁ,
as - ତା C₃ ବା ଡାହାଣ ପାର୍ଶ୍ୱରେ C₃
only C₁ / C₂ value pick ନିଅନ୍ତୁ
ନିଅନ୍ତୁ ।

S-attributed

- Based on synthesized att.
- use Bottom up parsing
- In Right most there semantic rules.

L-attributed

- only left sibling tree
- Based on synthesized of inherited attribute
 - Top Down parsing
 - Semantic rules can be anywhere.

S-attributed SDT :

1. If an SDT uses only synthesized attributes, it is called as S-attributed SDT.
2. S-attributed SDTs are evaluated in bottom-up parsing, as the values of the parent nodes depend upon the values of the child nodes.
3. Semantic actions are placed in rightmost place of R.H.S.

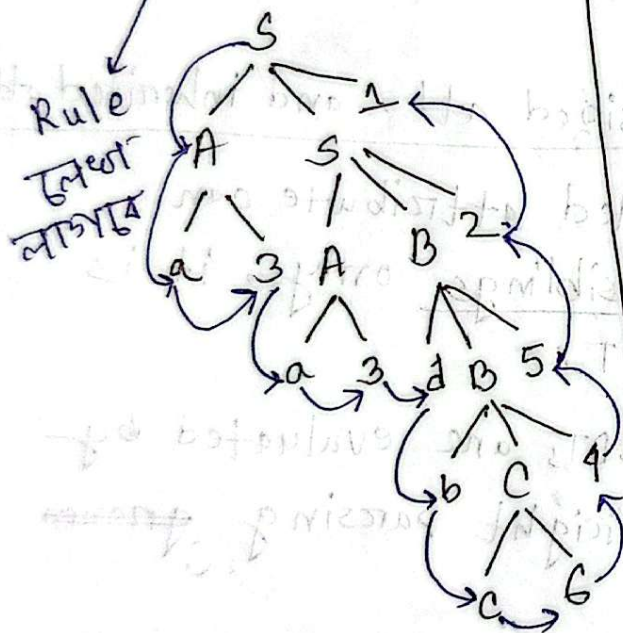
L-attributed SDT :

1. If an SDT uses both synthesized att. and inherited att. with a restriction that inherited attribute can inherit only values from left siblings only, it is called as L-attributed SDT.
2. Attributes in L-attributed SDTs are evaluated by depth-first and left-to-right parsing manner.
3. Semantic rules can be anywhere in R.H.S.

$S \rightarrow AS$ { printf (1) }
 $S \rightarrow AB$ { printf (2) }
 $A \rightarrow a$ { printf (3) }
 $B \rightarrow bC$ { printf (4) }
 $B \rightarrow dB$ { printf (5) }
 $C \rightarrow c$ { printf (6) }

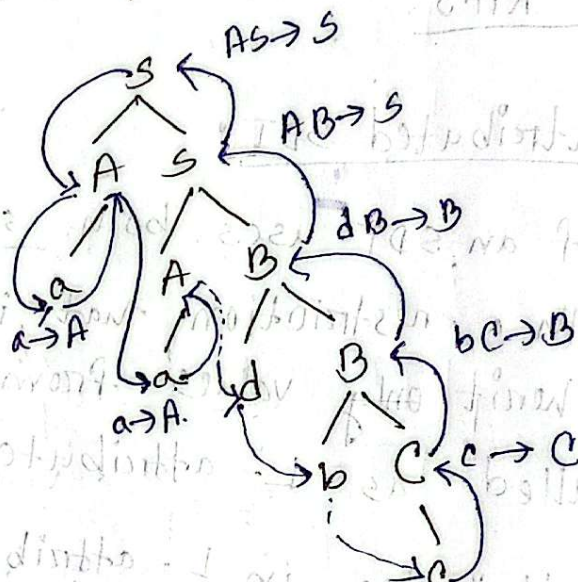
string : aadbc

Top down Approach



print : 3364521

Bottom up : rule लगाया



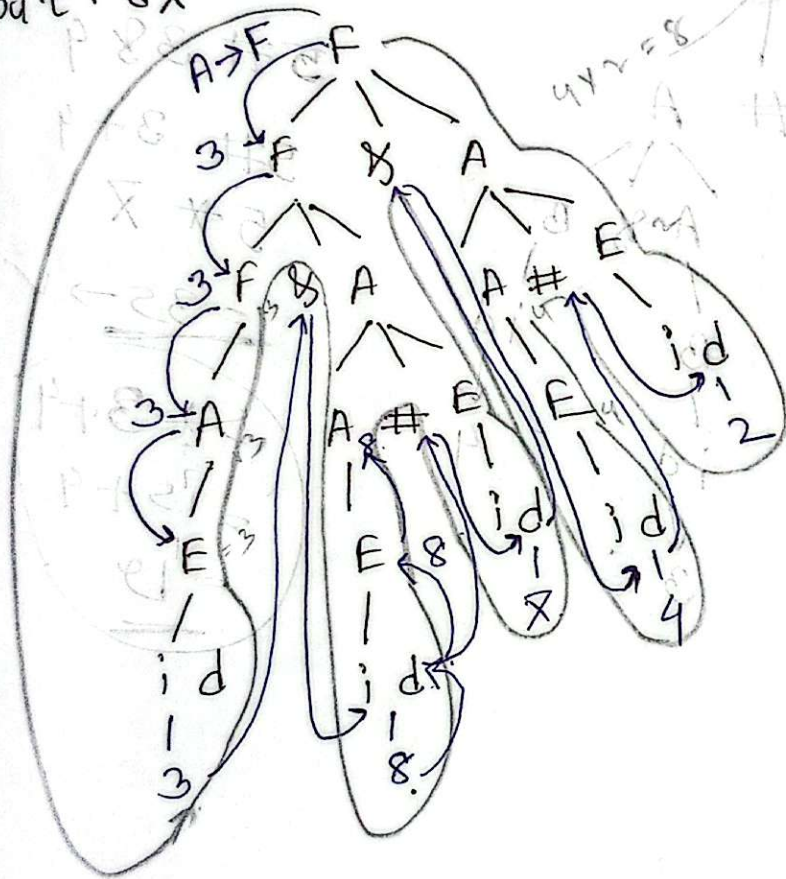
print : 3364521

2.

$$A \rightarrow A \# E \wedge A \cdot \text{val} \neq A \cdot \text{val} \wedge E \neq \text{val}$$
$$E \rightarrow id \in E, val = id \}$$

input: 3 8 7 4 2

output t: 6X

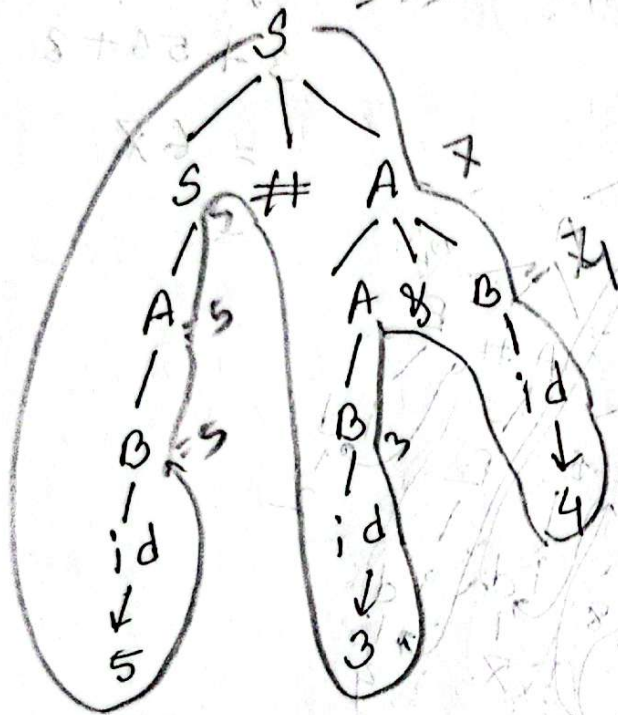


385688

$$3 + 56 + 8$$
$$= 67$$
$$\begin{aligned} A \cdot \text{val} \times E \cdot \text{val} \\ = 8 \times 8 \\ = 64 \end{aligned}$$

3. $S \rightarrow S \# A \mid A \quad \{ S.val = S.val * A.val; \}$
 $A \rightarrow A \& B \mid B \quad \{ A.val = A.val + B.val; \}$
 $B \rightarrow id \quad \{ B.val = id.lexical_value \}$

Given string : 5#3&4



$$\begin{aligned}
 &5 \# 3 \& 4 \\
 &5 \# 3 + 4 \\
 &5 * 7 \\
 &= 35 \rightarrow \checkmark \\
 &\text{---} \\
 &5 * 3 + 4 \\
 &= 15 + 4 \\
 &= 19
 \end{aligned}$$

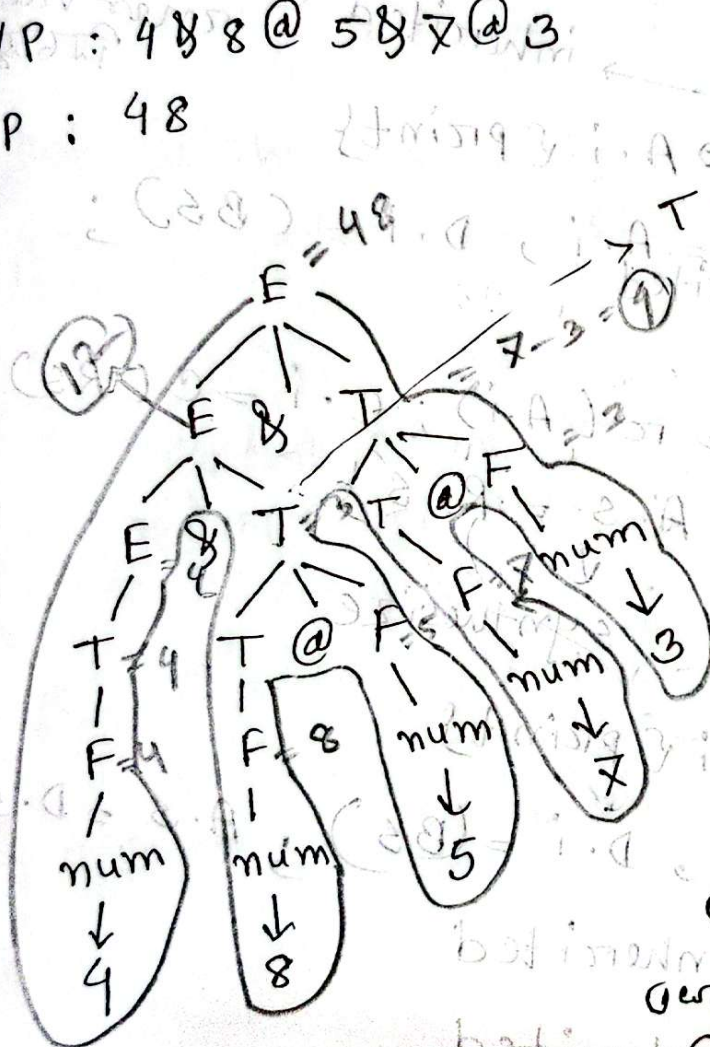
4. $E \rightarrow E \& T \{ E.val = E.val * T.val \}$
 $IT \{ E.val = T.val \}$

$T \rightarrow T @ F \{ T.val = T.val - F.val \}$
 $IF \{ T.val = F.val \}$

$F \rightarrow num \{ F.val = num \}$

I/P : $4 \& 8 @ 5 \& 7 @ 3$

O/P : 48



$T = T.val - F.val$
 $= 8 - 5$
 $= 3$

$E \rightarrow E \& T$
 $E.val * T.val$
 $12 * 4$
 $= 48$

Shortcut

first two symbol
 or priority for
 $\&$ and $@$
 @ first then $\&$
 priority for.

So, $4 \& 8 @ 5 \& 7 @ 3$

$4 \& 3 \& 8 \& 7$

$4 * 3 * 8$

$= 48$

$[\& \rightarrow *]$
 $[@ \rightarrow -]$

5. Grammar tree and its value
SDT & its value?

Step 1: Identify the inherited & synthesized

Step 2: Identify the SDT

Semantic rules $\rightarrow$ inherited (child parent value)

$A \rightarrow BD \{ D.i = A.i \text{ \& print } \}$

$A \rightarrow BD \{ B.i = A.i, D.i = (BS) \}$

$A.s = D.s$
 $\downarrow$
synthesized

$A \rightarrow CR \{ R.i = r(A.i); C.i = C(R.s);$

$A.s = R.s$
 $\downarrow$
synthesize

$\Rightarrow$

$A \rightarrow BD \{ D.i = A.i \text{ \& print } \}$

$\{ B.i = A.i, D.i = (BS), A.s = D.s \}$

$D.i = A.i \rightarrow$ inherited

$B.i = A.i \rightarrow$ inherited

$D.i = B.s \rightarrow$ inherited

$A.s = D.s \rightarrow$ synthesized

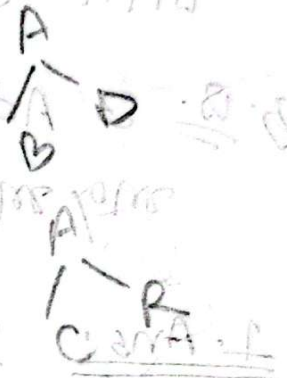
$A \rightarrow CR : (\text{synthesize, inherited both})$

$R.i = \pi(A.i) \rightarrow \text{Inherited}$

$C.i = \sigma(R.s) \rightarrow \text{Inherited } \times$

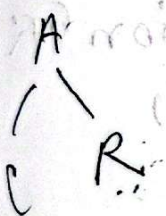
$A.s = R.s \rightarrow \text{synthesized}$

$A \rightarrow CR$ neither s-attributed /
non L-attributed SDT.



Restriction: इन-हील inherited

इन्हील non इल



$A \rightarrow B D \rightarrow L\text{-attributed.}$

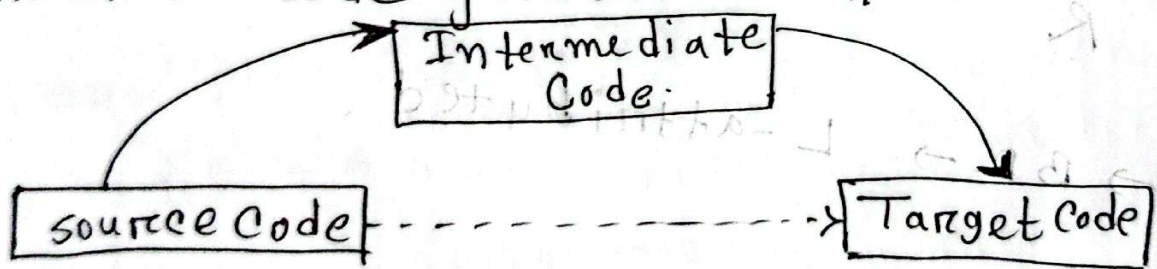


Q.1.8. Intermediate code generation work, why it is needed?

2.8. Explain the methods of Intermediate code generation how it's work with example.

3.8. 3 Address Code for statement for any example?

1. Ans: Intermediate code generation is a compiler technique that takes source program as input and generates machine-independent intermediate representation (IR). Intermediate representation is a representation of the source program that is used for optimization and machine code generation.



Working process:

1. The parser generates a syntax tree from the source code.

2. The tree is traversed to break complex expressions into simple expression operations.

3. Each operation is converted into intermediate code such as Three Address code (TAC), Quadruples, or Triples.

4. Temporary variables are used to store intermediate results.

example :

$$a = b + c * d$$

Intermediate code :

$$t_1 = c * d$$

$$t_2 = b + t_1$$

$$a = t_2$$

Why needed :

1. It makes compiler design easier and more modular.
2. It produces machine-independent code.
3. It helps to code optimization.
4. It simplifies debugging and error handling.

5. The same intermediate code can be used for different machine architectures.

2. Ans: The methods of intermediate code generation -

1. Postfix Notation (Reverse Polish notation)
2. Directed Acyclic Graph (DAG)
3. Syntax Tree (Parse tree)
4. Three-Address code (TAC)

1. Postfix Notation: The operator comes after the operands.

Example: Infix: $a + b$

postfix: $ab +$

Work

1. First change the infix expression into postfix form.
2. Read the postfix expression from left to right.
3. Use a stack:
 - If see a number or variable $\rightarrow$ push it
 - If see an operator $\rightarrow$ pop two values, perform the operation, push the result.

Example :

Infix expression : $a = b + c * d$

postfix : $b c d * +$

Intermediate code : $t_1 = c * d$

$t_2 = b + t_1$
 $a = t_2$

Infix : $a + (b * c + b * c + d)$

$\rightarrow a + (\underline{b * c} + \underline{b * c} + d)$

$\rightarrow a + (\underline{b c *} + \underline{b c *} + d)$

$\rightarrow a + (\underline{b c * b c *} + \underline{d})$

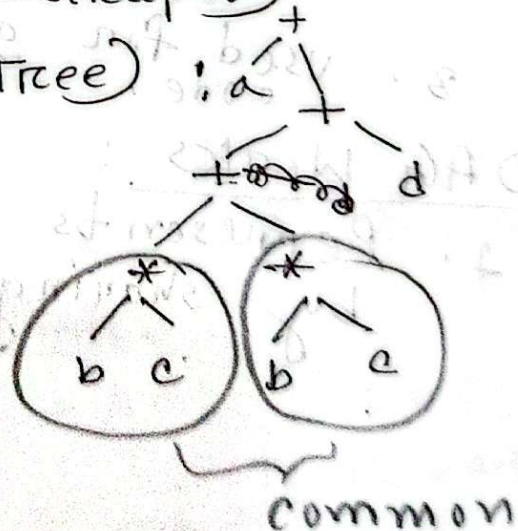
$\rightarrow \underline{a + (b c * b c * + d)}$

Postfix $\rightarrow a b c * b c * + d + +$

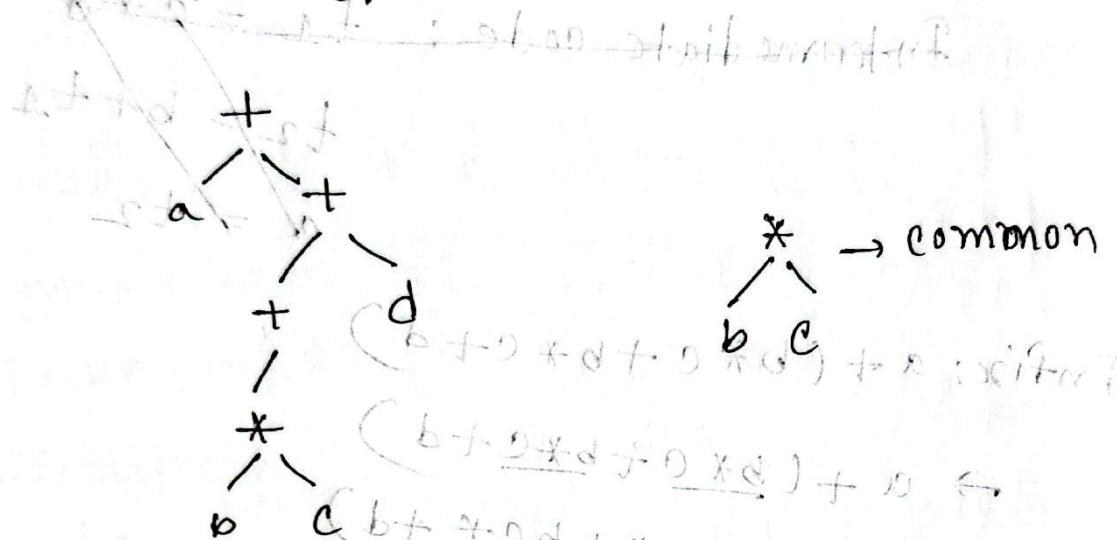
3 DAG (Directed Acyclic Graph)

Syntax Tree (parse Tree)

$a + (b * c + b * c + d)$



2. DAG : sub-expression syntax tree to common value किता. मेवा, मातर, common गुलर remove मरु केनेयु अहित मरु।



Works of Syntax tree :

1. Shows structure of expressions according to grammar.
2. Helps preserve operator precedence and associativity.
3. Used for generating intermediate code.

DAG Works :

1. Represents expressions efficiently by sharing nodes.

2. Eliminates common subexpressions to avoid repeated calculation.
3. helps generate optimized intermediate code.

4. Three-Address Code (TAC): is an intermediate representation in compiler design that simplifies -

complex high-level expression by breaking them down into sequence of simple instruction in TAC contain at most three-address (operands), typically involving one operator and two operands (or addresses) and a result.

example: $a + (b * c + b * c + d)$

3rd add/variables

$$T_1 = b * c$$

$$T_2 = b * c$$

$$T_3 = T_1 \oplus T_2$$

$$T_4 = T_3 \oplus d$$

$$T_5 = a \oplus T_4$$

common operator 2nd operand

optimize:

$$T_1 = b * c$$

$$T_3 = T_2 + d$$

$$T_2 = T_1 + T_1 \quad T_4 = a + T_3$$

remove the underline with 1

3. Ans: Three address code for for statement like

$$x = (a * b) + (c * d)$$

$$\begin{aligned} t_1 &= a * b \\ t_2 &= c * d \\ x &= t_1 + t_2 \end{aligned}$$

Various statements in 3-address code

1. Assignment $\rightarrow x = y$ (Binary) operation $\rightarrow x = y + z$

$\rightarrow x = y$ operation (Unary)

$\rightarrow x = y$ (direct operator assign) $\rightarrow x = y$

2. Jump $\rightarrow$ conditional $\rightarrow$ if x rel op y goto L

$\rightarrow$ unconditional $\rightarrow$ goto L

3. Array Assignment $\rightarrow x = y[i]$

$\rightarrow x[i] = y$

4. Pointer, address Assign $\rightarrow x = \&y$

$\rightarrow x = *y$

example: $a + \left(\frac{b * c + b * c + d}{t_1} \right)$

step 1: Assignment statement.

$$t_1 = b * c$$

$$t_2 = t_1 + t_1 \quad [\because b * c + b * c]$$

$$t_3 = t_2 + d$$

$$t_4 = a + t_3$$

step 2: Jump statement

Let, $a > 0$

if $a > 0$ goto L1

goto L2

step 3: Array Assignment

$$t_1 = b[i] * c[j]$$

$$arr[k] = t_1$$

step 4: pointer Assignment

$$t_1 = *p$$

$$*q = t_1$$

1.1. Loop optimization → important (summary type question)

1. Code Motion (Frequency Reduction):

- (high frequency reason for low frequency reason)
o convert/zero shift
- memory saving & time saving o & time
- Semantics of the program don't change
(code & structure, the order of statement etc change not affect)

```
1. a = 100;  
2. while (a > 0)  
3. {  
4. x = y + z;  
5. if (a % x == 0)  
6. printf("%d", a);  
7. }
```

Here, $x = y + z$ is calculated n times, even though y and z never change. So, we moved outside the loop.

↓

```
a = 100;  
x = y + z;  
while (a > 0)  
{  
    if (a % x == 0)  
        printf("%d", a);  
}
```

loop thus reduce the frequency of the code.

before optimization;

```
for (i=0; i<n; i++) {  
    x = a + b;  
    y[i] = x * i;  
}
```

Here, $x = a + b$; is calculated n times, even though 'a' and 'b' never change.

after optimization:

```
x = a + b;  
for (i=0; i<n; i++) {  
    y[i] = x * i;  
}
```

Code motion is a loop optimization technique that moves loop-invariant computations outside the loop to reduce execution frequency without changing program semantics.

2. Loop fusion:

- merging two or more loops that run over the same range.

- To reduce loop overhead and improve cache performance.

- semantics of the program do not change.

before:

```
int i, a[100], b[100]
for (i = 0; i < 100; i++)
    a[i] = 1;
for (i = 0; i < 100; i++)
    b[i] = 2;
```

X →

১. remove
২. ১ম লুপ
৩. ২য় লুপ
৪. merge
৫. লুপ

after:

```
int i, a[100], b[100]
for (i = 0; i < 100; i++)
    a[i] = 1;
    b[i] = 2;
```

Loop fusion is an optimization technique that combines multiple loops with the same bounds into a single loop to reduce overhead without changing program semantics.

3. loop unrolling : $\rightarrow$ loop ko kam krakar, multiple baar likh kr

$\rightarrow$ writing the loop body multiple times in one iteration

$\rightarrow$ so the loop runs fewer times

$\rightarrow$ This makes the program faster.

$\rightarrow$ But the meaning / output (semantics) stays the same.

Before :

```
for (i = 0; i < 8; i++) {  
    a[i] = b[i] + 1;  
}
```

after :

```
for (i = 0; i < 8; i += 4) {  
    a[i] = b[i] + 1;  
    a[i+1] = b[i+1] + 1;  
    a[i+2] = b[i+2] + 1;  
    a[i+3] = b[i+3] + 1;  
}
```

```
for (i = 0; i < 8; i++) {  
    printf("CSE-413");  
}
```

$i < 8$ ko 3 baar krke
usko CSE-413 (3 baar)
print krke loop use
kr krke same
output nikal skte hai

$\downarrow$

```
printf("CSE-413");  
printf("CSE-413");  
printf("CSE-413");
```

Loop optimization needed because -

1. Loops run many times, so even small improvements save a lot of time.
2. It makes the program faster.
3. It reduces CPU work and improves performance.
4. It removes unnecessary repeated work.
5. It improves memory and cache usage.

12. Peephole optimization :

~~***~~

- large part of code or small part of code
- removing unnecessary or redundant instructions
- Making the code faster and smaller.
- without changing the meaning.

1. Redundant load and store :

Redundant $\rightarrow$ ଅବ୍ୟବହାରୀ

Load $\rightarrow$ memory ରୁ Register - ୮ ଆସି ।

store $\rightarrow$ Register ରୁ memory ୮ ଯାଏ ।

ଏହା compiler ଦ୍ଵାରା ହେଉଛି -

1. ଆମେ ମାନି memory ୮ ଯାଏ (store)

2. ଅନ୍ୟତ୍ର ଯାଏ ଆମେ ଆମେ ମାନି memory

୮ ରୁ (load)

ତେଣୁ ଦୁଇଟି କାର୍ଯ୍ୟ ପରସ୍ପର ବିରୋଧୀ - ତେଣୁ ଏହା

Redundant load and store.

ex. $a = b + c$

$d = a + e$

Mov b, R0

Add c, R0

Mov R0, a

~~Mov a, R0~~

Add e, R0

Mov R0, d

ତେଣୁ instruction ର ଗୁଡ଼ିକ ଦିଆଯାଇଛି
ଯାହା ମଧ୍ୟ already a store
ଆଉ 2nd ଯେଉଁଠି ତେଣୁ ଆଉ load
କରାଯାଇ ନାହିଁ ।

2. Strength Reduction: where the compiler replaces a slow or expensive operation (like multiplication or division) with a faster operation (like subtraction, addition, or bit shifting) without changing the result.

before:

ex- $x = y * 8;$

after:

$x = y << 3;$ // left shift by 3 = multiply by 8

before:

$x = y * 2;$

after:

$x = y + y;$

before:

$x = y / 2;$

after:

$x = y >> 1;$ // right shift = divide by 2

$x^2 \Rightarrow x * x$

$x * 2 \Rightarrow$ left shift

$x / 2 \Rightarrow$ Right shift

→ makes the program run faster

→ Reduces CPU workload

→ Improves overall performance

3. Simplify Algebraic Expressions : where the

compiler simplifies mathematical expressions ^(+0, *1, /1, -0) to remove unnecessary operations, without changing the result.

ex: $x = y + 0$; after: $x = y$;

before: $x = y * 1$; after: $x = y$;

before: $x = y * 0$; after: $x = 0$;

before: $x = y + y$; after: $x = 2 * y$;

before: $x = y / 1$; after: $x = y$

~~$a := a + 0$~~

~~$a := a * 1$~~

~~$a := a / 1$~~

~~$a := a - 0$~~

↓

unnecessary

→ Reduces unnecessary operations

→ makes code faster and cleaner

→ Improves performance

4. Replace slower Instructions with faster :

the compiler replaces slow machine instructions with faster equivalent instructions, without changing the ~~pro~~ program's result.

Before (slow)

MUL R1, R2;

↓

multiply slow

after (faster)

SHL R1, 1,

↓

shift left faster

$X \text{ Add } \#1, R \Rightarrow \text{INC } R \checkmark$ (fast)
 $X \text{ Sub } \#1, R \Rightarrow \text{DEC } R \checkmark$

In Java Byte code

$\left. \begin{array}{l} a \text{ load } x \\ a \text{ load } x \\ \text{MVL} \end{array} \right\} \Rightarrow \begin{array}{l} a \text{ load } x \\ \text{dup} \\ \text{MVL} \end{array} \quad \left(\begin{array}{l} \text{duplicate} \\ \text{slow code to fast} \end{array} \right. \text{ use } m?$

- makes execution faster
- reduces instruction count
- improves CPU efficiency

5. Dead Code Elimination; compiler optimization

technique where remove such G code

that is - never execute
- execute शब्द का Result एम ७३
गुणांक मिला रहा है।

otherwise constant propagant

```

int dead (void)
{
    int a = 10;
    int b = 20;
    int c;
    * ← c = a * b;
    return c;
}

```

```
b = 30;  
b = b * 10;  
return b;  
}
```

6. → This is
 dead code
 ଆମର ଏହି part
 ରେ ଆମର code
 କି ବ୍ୟବହାର use
 ନୁହେଁ । So remove
 ଆମର ଏହି part.